

Advanced SQL Analytics using Superstore Dataset

Objective

In this project, I used the Superstore dataset to practice advanced SQL concepts. I worked with Subqueries, Common Table Expressions (CTEs), and Window Functions to answer different business questions and understand customer sales performance.

Step 1: Setup Data

First, I created a new database in MySQL and imported the Superstore dataset into a table named **superstore_raw**. After importing the data, I checked the table structure and verified that all records were loaded successfully.

Database create

Code:

```
CREATE DATABASE advanced_sql_db;
```

```
USE advanced_sql_db;
```

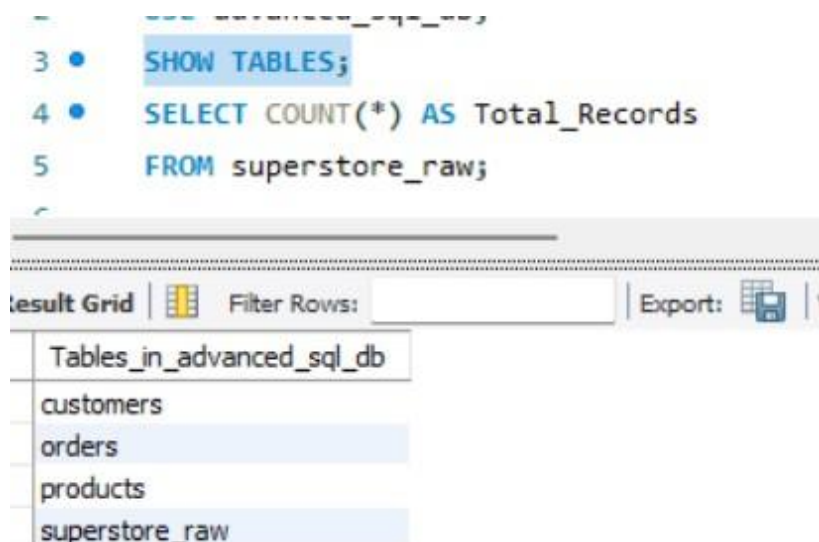
Output:

✓	3	14:29:01	CREATE DATABASE advanced_sql_db	1 row(s) affected	0.016 sec
---	---	----------	---------------------------------	-------------------	-----------

```
-- Check imported data
```

```
SHOW TABLES;
```

Output:



```
3 • SHOW TABLES;
4 • SELECT COUNT(*) AS Total_Records
5   FROM superstore_raw;
```

result Grid	Filter Rows:	Export:
Tables_in_advanced_sql_db		
customers		
orders		
products		
superstore_raw		

```
SELECT COUNT(*) AS Total_Records FROM superstore_raw;
```

Output:

```
4 • SELECT COUNT(*) AS Total_Records
5   FROM superstore_raw;
6
```

Result Grid	Filter Rows:	Export
Total_Records		
9694		

```
DESC superstore_raw;
```

Output:

```
7 • DESC superstore_raw;
8 • CREATE TABLE customers AS
```

Result Grid	Filter Rows:	Export:	Wrap Cell C		
Field	Type	Null	Key	Default	Extra
Row ID	int	YES		NULL	
Order ID	text	YES		NULL	
Order Date	text	YES		NULL	
Ship Date	text	YES		NULL	
Ship Mode	text	YES		NULL	
Customer ID	text	YES		NULL	
Customer Name	text	YES		NULL	
Segment	text	YES		NULL	
Country	text	YES		NULL	
City	text	YES		NULL	
State	text	YES		NULL	
Postal Code	int	YES		NULL	
Region	text	YES		NULL	
Product ID	text	YES		NULL	
Category	text	YES		NULL	
Sub-Category	text	YES		NULL	
Product Name	text	YES		NULL	
Sales	double	YES		NULL	
Quantity	int	YES		NULL	
Discount	double	YES		NULL	
Profit	double	YES		NULL	

Step 2: Create Customers, Orders and Products Tables

The raw dataset contains repeated customer, order, and product information. To make the data easier to work with, I created separate tables using **SELECT DISTINCT**.

-- Renamed the column names to make them short and easy to use

```
CREATE TABLE customers AS
SELECT DISTINCT
  `Customer ID` AS customer_id,
  `Customer Name` AS customer_name,
  Segment AS segment
FROM superstore_raw;
```

```
CREATE TABLE orders AS
SELECT DISTINCT
  `Order ID` AS order_id,
  `Order Date` AS order_date,
  `Ship Date` AS ship_date,
  `Ship Mode` AS ship_mode,
  `Customer ID` AS customer_id,
  Sales AS sales,
  Quantity AS quantity,
  Discount AS discount,
  Profit AS profit
FROM superstore_raw;
```

```
CREATE TABLE products AS
SELECT DISTINCT
  `Product ID` AS product_id,
  `Product Name` AS product_name,
  Category AS category,
  `Sub-Category` AS sub_category
FROM superstore_raw;
```

Step 3: Subqueries

In this step, I used subqueries to compare sales values with average sales and to find the highest sales order for each customer.

```
-- Q1. Find all orders where sales are greater than the average sales.
(Subquery)
SELECT *
FROM orders
WHERE sales >
(
  SELECT AVG(sales)
  FROM orders
);
```

Output:

```
52  -- Q1. Find all orders where sales are greater than the average sales.
53
54  • SELECT *
55  FROM orders
56  WHERE sales > (
57      SELECT AVG(sales)
58      FROM orders
59  );
60
```

Result Grid										Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	order_id	order_date	ship_date	ship_mode	customer_id	sales	quantity	discount	profit				
▶	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	261.96	2	0	41.9136				
	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582				
	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031				
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	907.152	6	0.2	90.7152				
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.184	9	0.2	85.3092				
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	911.424	4	0.2	68.3568				
	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	407.976	3	0.2	132.592				
	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	665.88	6	0	13.3176				
	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.63	3	0	240.264				
	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.05				
	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.473				
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	532.3992	3	0.32	-46.9764				
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	371.168	4	0.2	41.7564				
	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	1029.95	5	0	298.685				

-- Q2. Find the highest sales order for each customer. (Subquery)

```
SELECT *
FROM orders o
WHERE sales = (
    SELECT MAX(sales)
    FROM orders
    WHERE customer_id = o.customer_id
);
```

Output:

```
61  -- Q2. Find the highest sales order for each customer.
62
63  • SELECT *
64  FROM orders o
65  WHERE sales = (
66      SELECT MAX(sales)
67      FROM orders
68      WHERE customer_id = o.customer_id
69  );
70
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_id	order_date	ship_date	ship_mode	customer_id	sales	quantity	discount	profit
▶	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582
	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.184	9	0.2	85.3092
	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.63	3	0	240.264
	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.05
	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.473
	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	532.3992	3	0.32	-46.9764
	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	1029.95	5	0	298.685
	CA-2016-111682	6/17/2016	6/18/2016	First Class	TR-21055	319.41	5	0.1	7.098

Step 4: Common Table Expressions (CTEs)

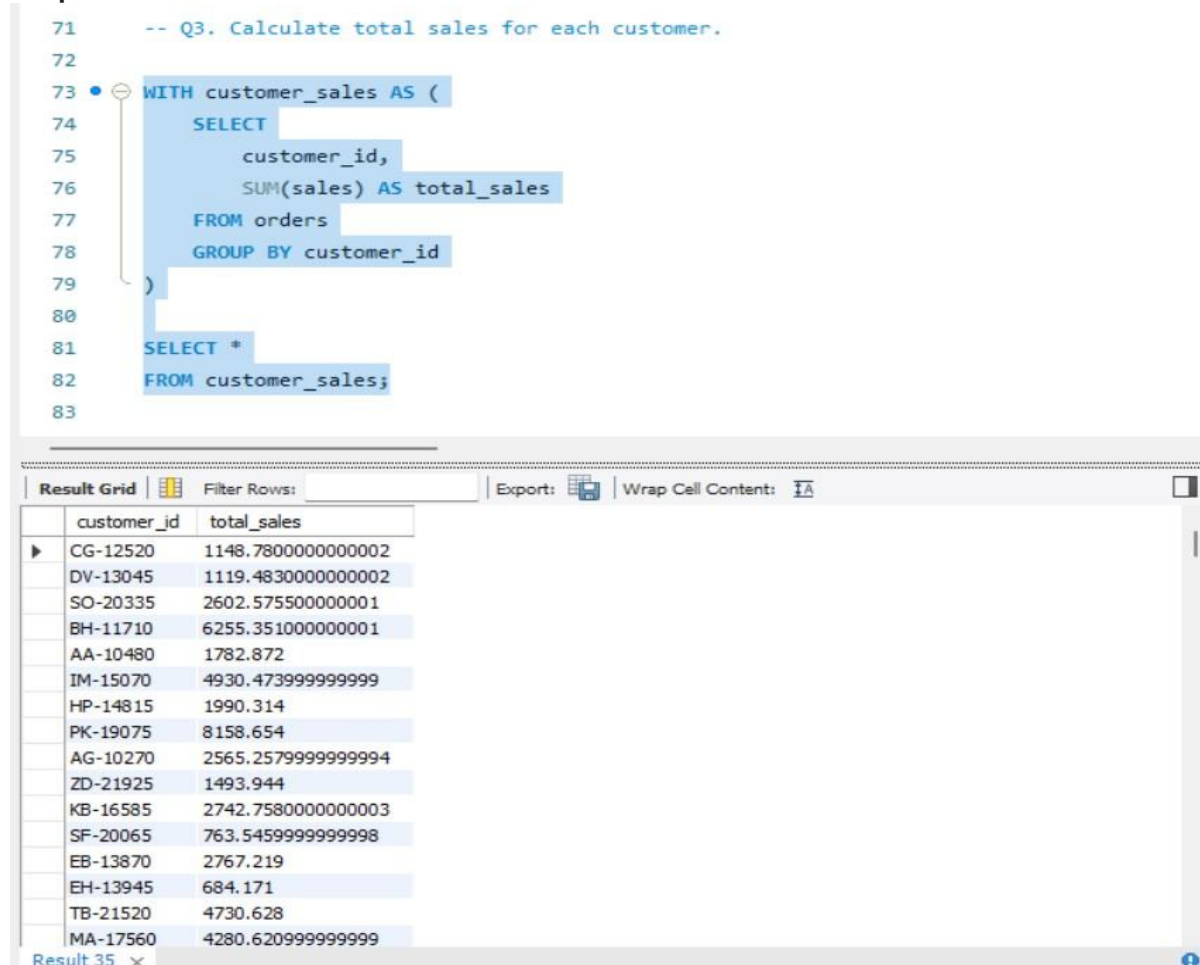
CTEs helped me calculate customer sales in a simple and readable way. They also made it easy to compare customer sales with the average sales.

```
-- Q3. Calculate total sales for each customer. (CTE)
```

```
WITH customer_sales AS (  
    SELECT  
        customer_id,  
        SUM(sales) AS total_sales  
    FROM orders  
    GROUP BY customer_id  
)
```

```
SELECT *  
FROM customer_sales;
```

Output:



```
71 -- Q3. Calculate total sales for each customer.  
72  
73 WITH customer_sales AS (  
74     SELECT  
75         customer_id,  
76         SUM(sales) AS total_sales  
77     FROM orders  
78     GROUP BY customer_id  
79 )  
80  
81 SELECT *  
82 FROM customer_sales;  
83
```

customer_id	total_sales
CG-12520	1148.7800000000002
DV-13045	1119.4830000000002
SO-20335	2602.5755000000001
BH-11710	6255.3510000000001
AA-10480	1782.872
IM-15070	4930.4739999999999
HP-14815	1990.314
PK-19075	8158.654
AG-10270	2565.2579999999994
ZD-21925	1493.944
KB-16585	2742.7580000000003
SF-20065	763.5459999999998
EB-13870	2767.219
EH-13945	684.171
TB-21520	4730.628
MA-17560	4280.6209999999999

```
-- Q4. Find customers whose total sales are above average. (CTE + Subquery)
```

```
WITH customer_sales AS (  
    SELECT  
        customer_id,  
        SUM(sales) AS total_sales  
    FROM orders  
    GROUP BY customer_id  
)
```



```

SELECT *
FROM customer_sales
WHERE total_sales > (
    SELECT AVG(total_sales)
    FROM customer_sales
);

```

Output:

```

84  -- Q4. Find customers whose total sales are above average.
85
86  WITH customer_sales AS (
87      SELECT
88          customer_id,
89          SUM(sales) AS total_sales
90      FROM orders
91      GROUP BY customer_id
92  )
93
94  SELECT *
95  FROM customer_sales

```

Result Grid | Filter Rows: | Exports: | Wrap Cell Content: [IA](#)

	customer_id	total_sales
▶	BH-11710	6255.351000000001
	IM-15070	4930.473999999999
	PK-19075	8158.654
	TB-21520	4730.628
	MA-17560	4280.620999999999
	SN-20710	3127.9592000000007
	LC-16930	4481.162
	RA-19885	3832.3139999999994
	ES-14080	4648.516
	KM-16720	4909.472000000001
	KD-16270	8282.358
	JE-15745	6400.454999999999
	SC-20770	4492.656999999999
	CS-12400	3651.8585000000007
	GM-14455	3078.6240000000003
	RB-19465	4556.566000000002

Result 41 x

Step 5: Window Functions

Window functions helped me rank customers based on sales and assign row numbers to orders without changing the original data.

```

-- Q5. Rank all customers based on total sales. (Window Function)
SELECT
    customer_id,
    SUM(sales) AS total_sales,
    RANK() OVER (ORDER BY SUM(sales) DESC) AS customer_rank
FROM orders
GROUP BY customer_id;

```

Output:

```
101 -- Q5. Rank all customers based on total sales.
102
103 • SELECT
104     customer_id,
105     SUM(sales) AS total_sales,
106     RANK() OVER (ORDER BY SUM(sales) DESC) AS customer_rank
107 FROM orders
108 GROUP BY customer_id;
109
110 -- Q6. Assign row numbers to each order within a customer.
```

customer_id	total_sales	customer_rank
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3
TA-21385	14595.62	4
AB-10105	14355.610999999997	5
SC-20095	14142.333999999999	6
KL-16645	14071.917	7
HL-15040	12873.297999999999	8
SE-20110	12209.438000000002	9
CC-12370	12129.072	10
TS-21370	11885.871	11
GT-14710	11820.119999999997	12
BM-11140	11609.900000000001	13
SV-20365	11420.645999999997	14
CJ-12010	11079.742	15
CL-12565	10880.545999999998	16

-- Q6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)

```
SELECT
    customer_id,
    order_id,
    sales,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY order_id
    ) AS order_number
FROM orders;
```

Output:

```
110 -- Q6. Assign row numbers to each order within a customer.
111
112 • SELECT
113     customer_id,
114     order_id,
115     sales,
116     ROW_NUMBER() OVER (
117         PARTITION BY customer_id
118         ORDER BY order_id
119     ) AS order_number
120 FROM orders;
121
122
```

customer_id	order_id	sales	order_number
AA-10315	CA-2014-128055	52.98	1
AA-10315	CA-2014-128055	673.568	2
AA-10315	CA-2014-138100	14.56	3
AA-10315	CA-2014-138100	14.94	4
AA-10315	CA-2015-121391	26.96	5
AA-10315	CA-2016-103982	3930.072	6
AA-10315	CA-2016-103982	41.72	7
AA-10315	CA-2016-103982	431.976	8
AA-10315	CA-2016-103982	2.304	9
AA-10315	CA-2017-147039	362.94	10
AA-10315	CA-2017-147039	11.54	11
AA-10375	CA-2014-130729	34.272	1
AA-10375	CA-2014-158064	16.52	2
AA-10375	CA-2015-109939	5.248	3
AA-10375	CA-2015-114503	84.96	4
AA-10375	CA-2015-140921	149.97	5

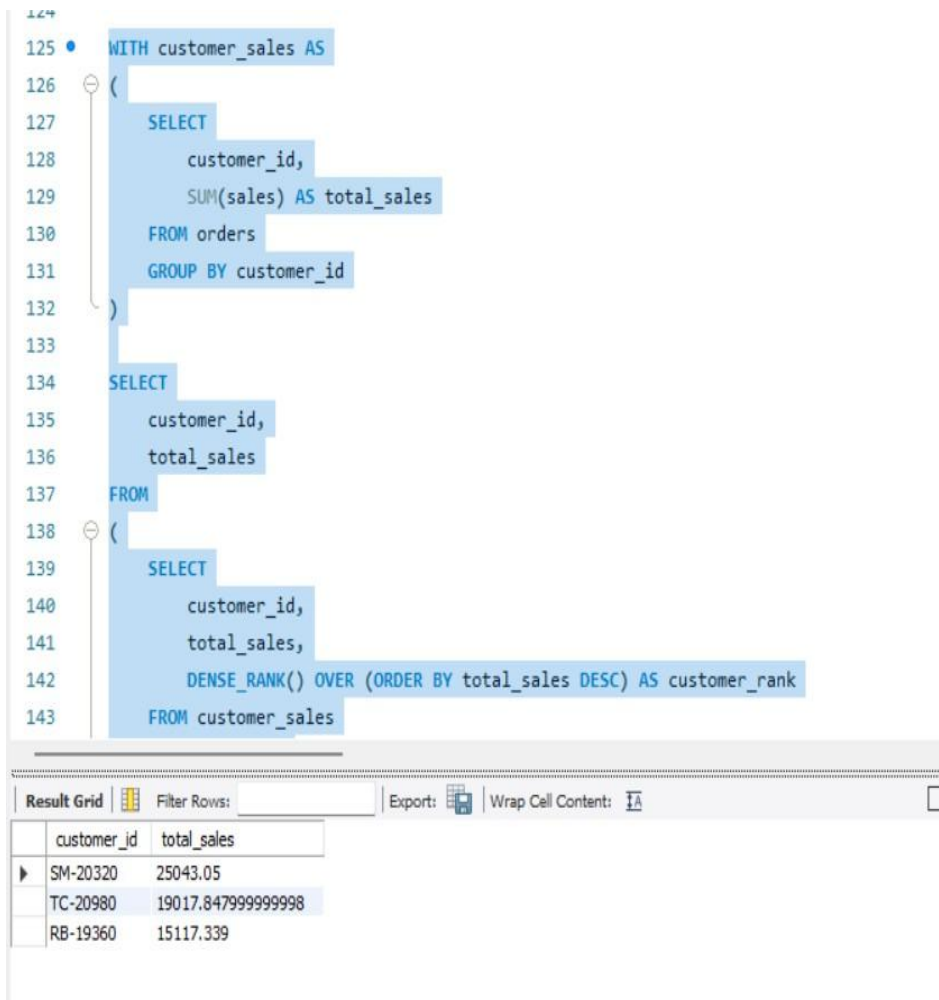
```

-- Q7. Display top 3 customers based on total sales. (Window Function)
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    total_sales
FROM
(
    SELECT
        customer_id,
        total_sales,
        DENSE_RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
    FROM customer_sales

) ranked_customers
WHERE customer_rank <= 3;

```

Output:



customer_id	total_sales
SM-20320	25043.05
TC-20980	19017.847999999998
RB-19360	15117.339

Step 6: Final Combined Query

In this query, I combined a JOIN, a CTE, and a Window Function to display customer name, total sales, and sales rank in a single result.

```
-- Who are the top 5 customers?
```

```
SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id
ORDER BY total_sales DESC
LIMIT 5;
```

Output:

```
169      -- Who are the top 5 customers?
170 •    SELECT
171      customer_id,
172      SUM(sales) AS total_sales
173      FROM orders
174      GROUP BY customer_id
175      ORDER BY total_sales DESC
176      LIMIT 5;
177
```

customer_id	total_sales
SM-20320	25043.05
TC-20980	19017.847999999998
RB-19360	15117.339
TA-21385	14595.62
AB-10105	14355.610999999997

```
-- Who are the bottom 5 customers?
```

```
SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id
ORDER BY total_sales
LIMIT 5;
```

Output:

```
178 -- Who are the bottom 5 customers?
179 • SELECT
180     customer_id,
181     SUM(sales) AS total_sales
182 FROM orders
183 GROUP BY customer_id
184 ORDER BY total_sales
185 LIMIT 5;
186
187 -- Which customers made only one order?
```

Result Grid	Filter Rows:	Export
customer_id	total_sales	
TS-21085	4.833	
LD-16855	5.304	
MG-18205	12.32	
CJ-11875	16.52	
RS-19870	22.328	

Result 49 x

-- Which customers made only one order?

```
SELECT
customer_id,
COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) = 1;
```

Output:

```
187 -- Which customers made only one order?
188 • SELECT
189     customer_id,
190     COUNT(order_id) AS total_orders
191 FROM orders
192 GROUP BY customer_id
193 HAVING COUNT(order_id) = 1;
194
195 -- Which customers have above-average sales?
196 • WITH customer_sales AS
```

Result Grid	Filter Rows:	Export:	Wrap Cell Conte
customer_id	total_sales		
TS-21085	4.833		
LD-16855	5.304		
MG-18205	12.32		
CJ-11875	16.52		
RS-19870	22.328		

Result 49 x

```
-- Which customers have above-average sales?
```

```
WITH customer_sales AS
(
SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id
)
```

```
SELECT *
FROM customer_sales
WHERE total_sales >
(
SELECT AVG(total_sales)
FROM customer_sales
);
```

Output:

```
195 -- Which customers have above-average sales?
196 • WITH customer_sales AS
197 (
198     SELECT
199         customer_id,
200         SUM(sales) AS total_sales
201     FROM orders
202     GROUP BY customer_id
203 )
204
205 SELECT
206     customer_id,
207     total_sales
208 FROM customer_sales
209 WHERE total_sales >
210 (
211     SELECT AVG(total_sales)
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

	customer_id	total_sales
▶	BH-11710	6255.351000000001
	IM-15070	4930.4739999999999
	PK-19075	8158.654
	TB-21520	4730.628
	MA-17560	4280.6209999999999
	SN-20710	3127.9592000000007
	LC-16930	4481.162
	RA-19885	3832.3139999999994
	ES-14080	4648.516
	KM-16720	4909.4720000000001
	KD-16270	8282.358
	TE-15745	5400.4540000000000

```
-- What is the highest order value per customer?
```

```
SELECT  
customer_id,  
MAX(sales) AS highest_order_value  
FROM orders  
GROUP BY customer_id;
```

Output:

```
215 -- What is the highest order value per customer?  
216 • SELECT  
217 customer_id,  
218 MAX(sales) AS highest_order_value  
219 FROM orders  
220 GROUP BY customer_id;  
221
```

customer_id	highest_order_value
CG-12520	731.94
DV-13045	721.875
SO-20335	957.5775
BH-11710	1706.184
AA-10480	479.97
IM-15070	1819.86
HP-14815	1242.9
PK-19075	4305.552
AG-10270	1336.44
ZD-21925	823.96
KB-16585	1013.832
SF-20065	254.9

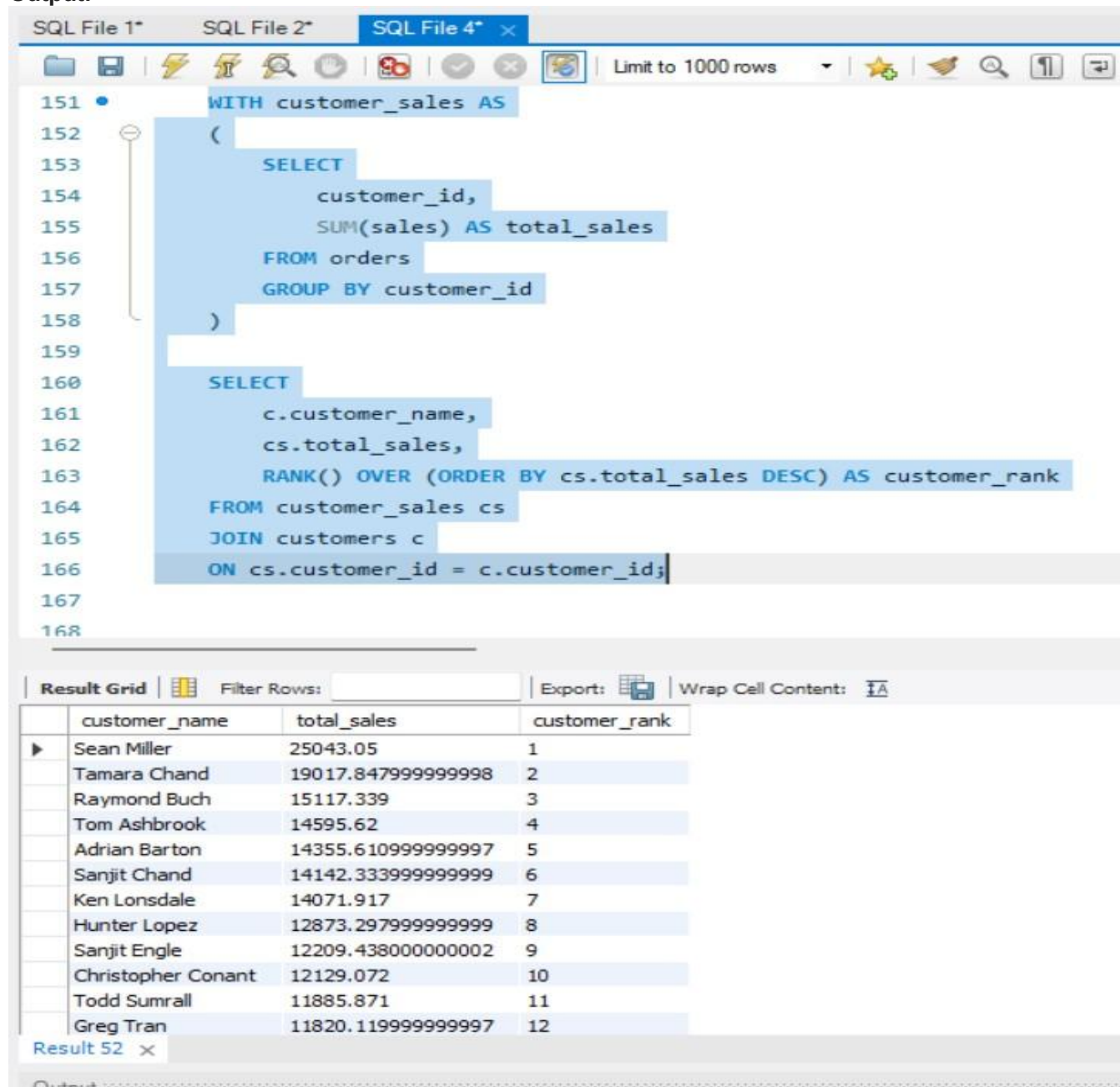
Combined Code:

```
-- Final Query  
-- Show customer name, total sales and rank
```

```
WITH customer_sales AS  
(  
    SELECT  
        customer_id,  
        SUM(sales) AS total_sales  
    FROM orders  
    GROUP BY customer_id  
)  
  
SELECT  
    customer_name,  
    total_sales,  
    RANK() OVER(ORDER BY total_sales DESC) AS customer_rank  
FROM customer_sales  
JOIN customers
```

```
ON customer_sales.customer_id = customers.customer_id;
```

Output:



The screenshot shows a SQL IDE with three tabs: 'SQL File 1*', 'SQL File 2*', and 'SQL File 4*'. The active tab 'SQL File 4*' contains a SQL query. The query uses a Common Table Expression (CTE) named 'customer_sales' to calculate total sales per customer from the 'orders' table. It then joins this CTE with the 'customers' table and uses the RANK() function to rank customers by their total sales in descending order. The results are displayed in a 'Result Grid' below the query editor.

```
151 WITH customer_sales AS
152 (
153     SELECT
154         customer_id,
155         SUM(sales) AS total_sales
156     FROM orders
157     GROUP BY customer_id
158 )
159
160 SELECT
161     c.customer_name,
162     cs.total_sales,
163     RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
164 FROM customer_sales cs
165 JOIN customers c
166 ON cs.customer_id = c.customer_id;
167
168
```

Result Grid

	customer_name	total_sales	customer_rank
▶	Sean Miller	25043.05	1
	Tamara Chand	19017.847999999998	2
	Raymond Buch	15117.339	3
	Tom Ashbrook	14595.62	4
	Adrian Barton	14355.610999999997	5
	Sanjit Chand	14142.333999999999	6
	Ken Lonsdale	14071.917	7
	Hunter Lopez	12873.297999999999	8
	Sanjit Engle	12209.438000000002	9
	Christopher Conant	12129.072	10
	Todd Sumrall	11885.871	11
	Greg Tran	11820.119999999997	12

Result 52 x

Key Insights

- The Superstore dataset contains **9694 records**, which were loaded successfully into the database.
- I created separate **customers**, **orders**, and **products** tables from the raw dataset using `SELECT DISTINCT`.
- I used a **Subquery** to find all orders where the sales value is higher than the average sales.
- Another subquery helped me identify the **highest sales order for each customer**.
- Using a **CTE**, I calculated the total sales of every customer, which made the queries easier to read.
- I found that only some customers have **total sales greater than the average customer sales**.
- With the help of **RANK()**, I ranked customers based on their total sales.

- I used **ROW_NUMBER()** with `PARTITION BY` to give each order a separate number for every customer.
- Finally, I combined **JOIN**, **CTE**, and **Window Functions** to display the customer name, total sales, and customer rank in a single query.